Data Structures & Algorithms

Recursion

Powerful alternate way of achieving repetition; occurs when a function refers to itself in its definition.

RECURSION

Recursive Thinking

- ***Recursion*** is:
  - A *problem-solving **approach***, that can ...
  - Generate *simple solutions* to ...
  - *Certain kinds* of problems that ...
  - Would be *difficult to solve in other ways*
- Recursion *splits a problem*:
  - Into one or more *simpler versions of **itself***

Recursive Thinking: The General Approach

1. if problem is "*small enough*"
2. solve it *directly*
3. else
4. break into one or more *smaller subproblems*
5. solve each subproblem *recursively*
6. *combine* results into solution to whole problem

Factorial Function

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1 & \text{if } n \geq 1. \end{cases}$$

Factorial

$$\text{factorial}(5) = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) = 5 \cdot \text{factorial}(4).$$

Recursive Definition of Factorial Function

Base case

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{factorial}(n-1) & \text{if } n \geq 1. \end{cases}$$

Recursive case

Recursive Implementation of Factorial

```
int recursiveFactorial(int n) {                  // recursive factorial function
  if (n == 0) return 1;                          // basis case
  else return n * recursiveFactorial(n−1);       // recursive case
}
```

```
int recursiveFactorial(int n) {
  if (n == 0) return 1;
  else return n * recursiveFactorial(n−1);
}
```

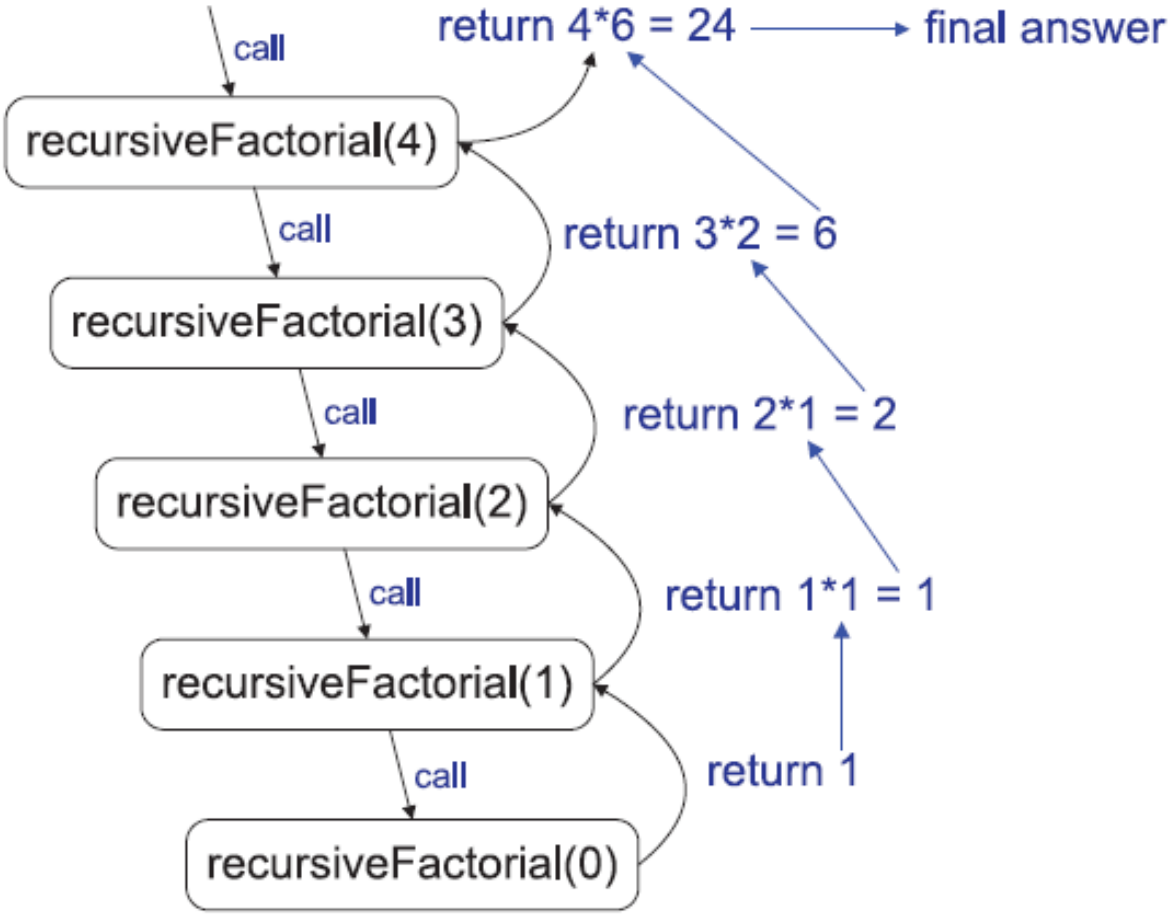


Figure 3.16: A recursion trace for the call recursiveFactorial(4).

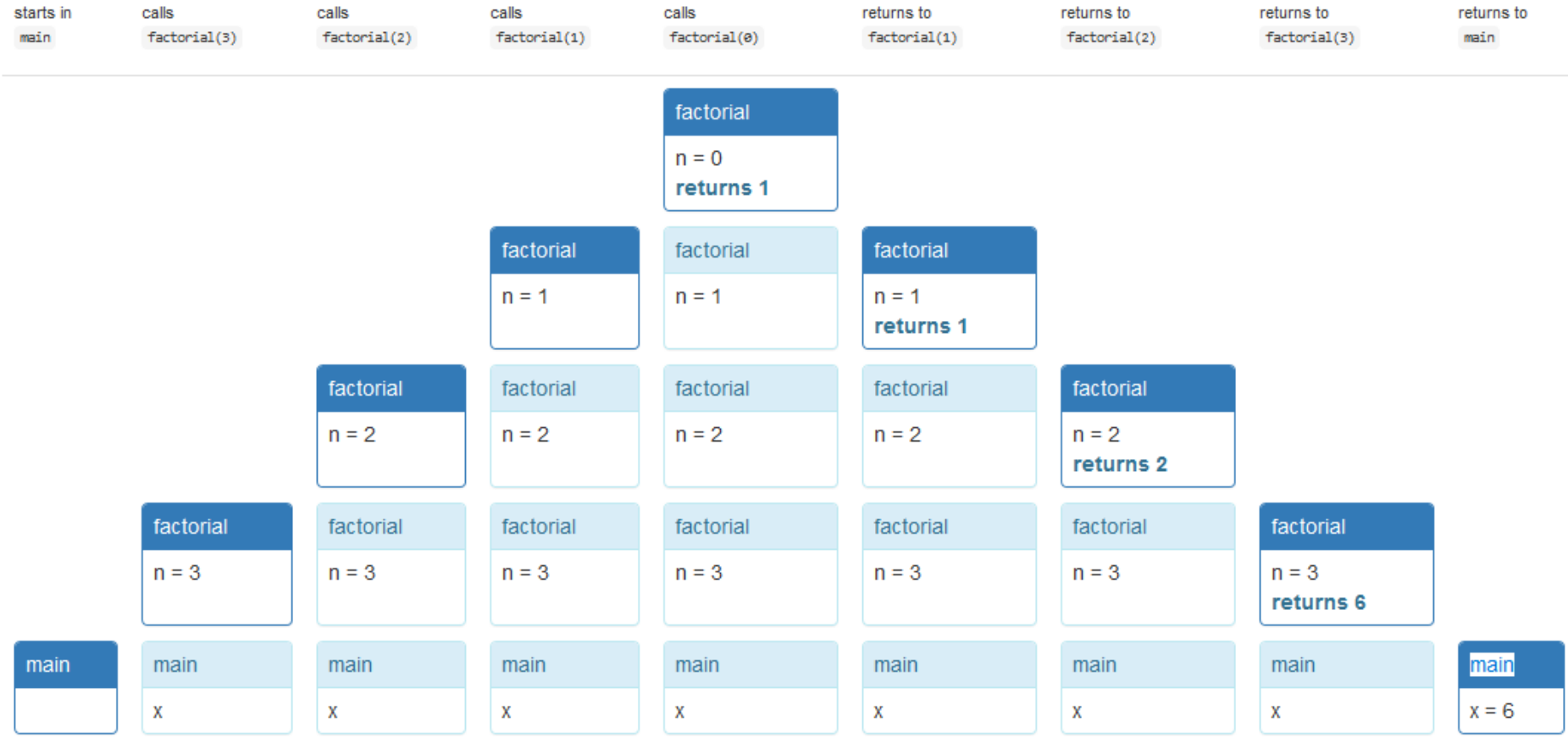
HELPED AVOID COMPLEX CASE ANALYSES AND NESTED LOOPS

MORE READABLE, BUT STILL EFFICIENT ALGORITHM DESCRIPTIONS

```
public static void main(String[] args) {
    long x = factorial(3);
}
```

```
public static long factorial(int n) {
  if (n == 0) {
    return 1;
  } else {
    return n * factorial(n-1);
  }
}
```

At each step, with time moving left to right:



https://web.mit.edu/6.005/www/fa16/classes/14-recursion/

The simplest form of recursion is ***linear recursion***, where a function is defined so that it makes at most one recursive call each time it is invoked.

LINEAR RECURSION

Summing the Elements of an Array Recursively

Algorithm LinearSum(A,n):

Input: A integer array A and an integer $n \geq 1$, such that A has at least n elements

Output: The sum of the first n integers in A

```
if n = 1 then
    return A[0]
else
    return LinearSum(A, n − 1) + A[n − 1]
```

Code Fragment 3.38: Summing the elements in an array using linear recursion.

Linear Recursion

- ***Test for base cases.*** We begin by testing for a set of base cases (there should be at least one). These base cases should be defined so that every possible chain of recursive calls eventually reaches a base case, and the handling of each base case should not use recursion.
- ***Recur.*** After testing for base cases, we then perform a single recursive call. This recursive step may involve a test that decides which of several possible recursive calls to make, but it should ultimately choose to make just one of these calls each time we perform this step. Moreover, we should define each possible recursive call so that it makes progress towards a base case.

Algorithm LinearSum(A,n):

Input: A integer array A and an integer $n \geq 1$, such that A has at least n elements

Output: The sum of the first n integers in A

if $n = 1$ **then**
  return $A[0]$
else
  return LinearSum$(A,n-1)+A[n-1]$

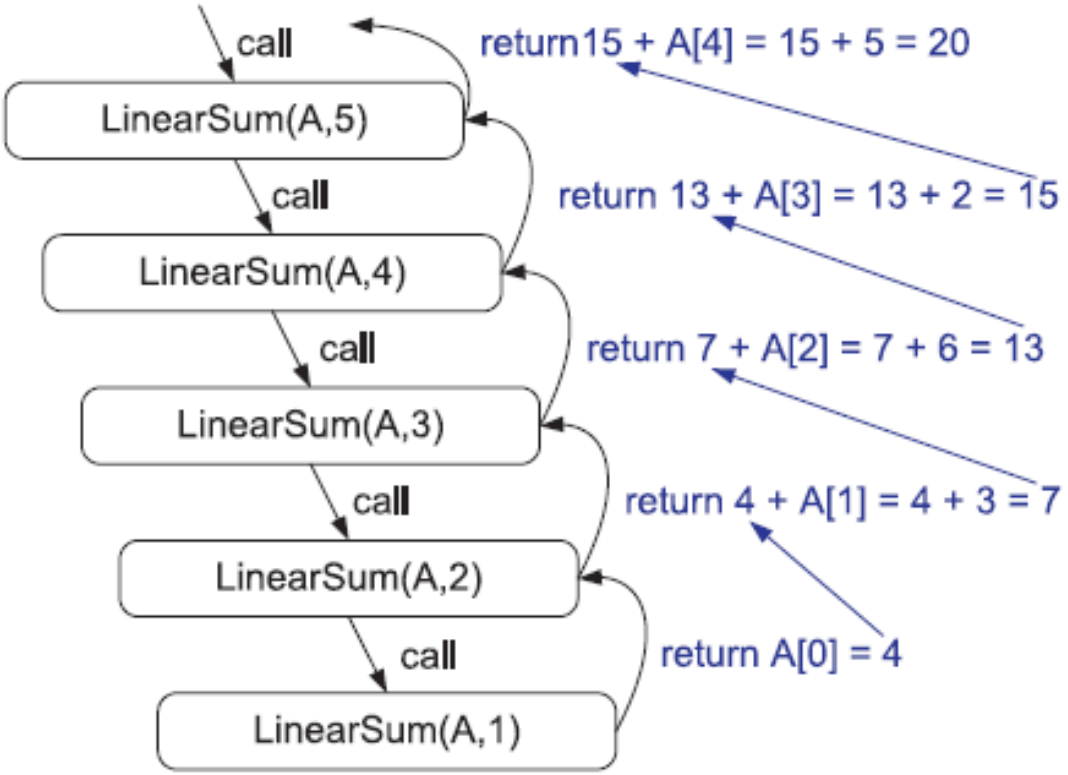


Figure 3.19: Recursion trace for an execution of LinearSum(A,n) with input parameters $A = \{4,3,6,2,5\}$ and $n = 5$.

Reversing an Array by Recursion

Algorithm ReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

if $i < j$ **then**

  Swap $A[i]$ and $A[j]$

  ReverseArray$(A, i+1, j-1)$

return

Code Fragment 3.39: Reversing the elements of an array using linear recursion.

Algorithm IterativeReverseArray(A, i, j):

Input: An array A and nonnegative integer indices i and j

Output: The reversal of the elements in A starting at index i and ending at j

while $i < j$ **do**

Swap $A[i]$ and $A[j]$

$i \leftarrow i + 1$

$j \leftarrow j - 1$

return

Code Fragment 3.40: Reversing the elements of an array using iteration.

When an algorithm makes two recursive calls, we say that it uses ***binary recursion***.

BINARY RECURSION

Algorithm BinarySum(A, i, n):

Input: An array A and integers i and n

Output: The sum of the n integers in A starting at index i

if $n = 1$ **then**

  return $A[i]$

return BinarySum$(A, i, \lceil n/2 \rceil)$ + BinarySum$(A, i + \lceil n/2 \rceil, \lfloor n/2 \rfloor)$

Code Fragment 3.41: Summing the elements in an array using binary recursion.

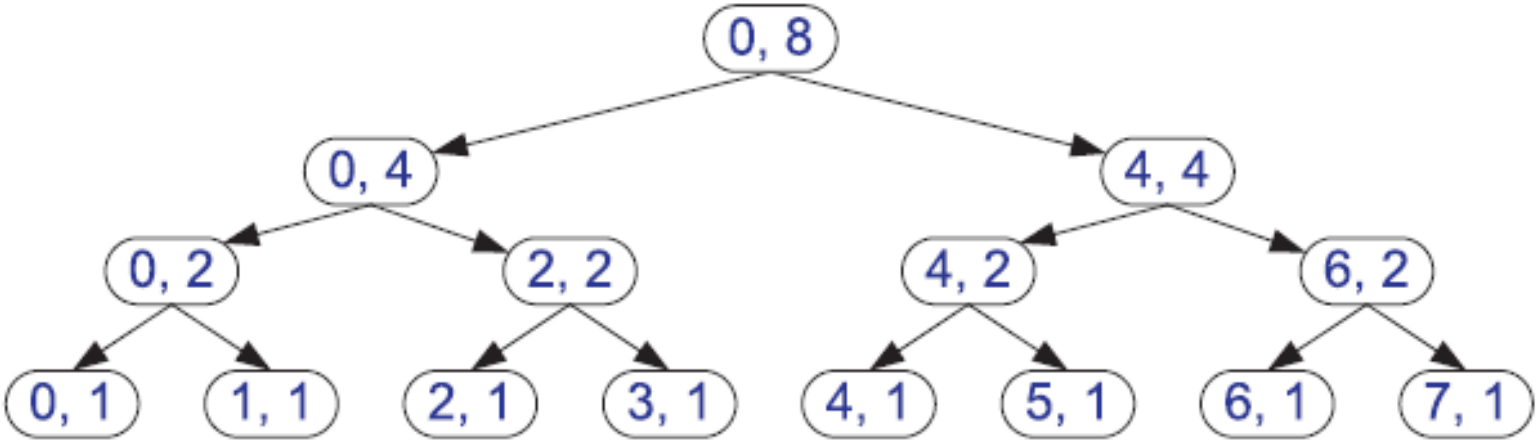


Figure 3.20: Recursion trace for the execution of BinarySum$(0, 8)$.

Generalizing from binary recursion, we use ***multiple recursion*** when a function may make multiple recursive calls, with that number potentially being more than two.

MULTIPLE RECURSION

Problem Solving with Recursion

- Towers of Hanoi
- Counting grid squares in a blob
- Backtracking, as in maze search

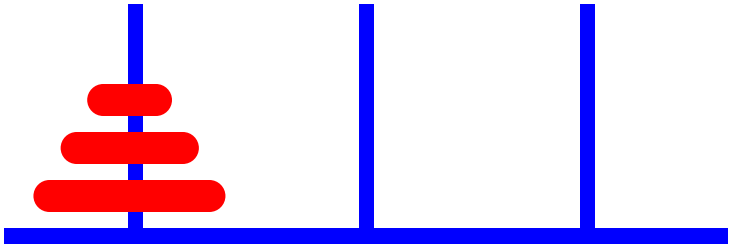
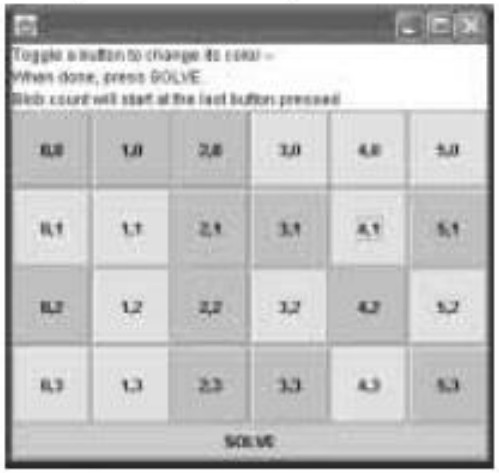


FIGURE 7.16
A Sample Grid for Counting Cells in a Blob

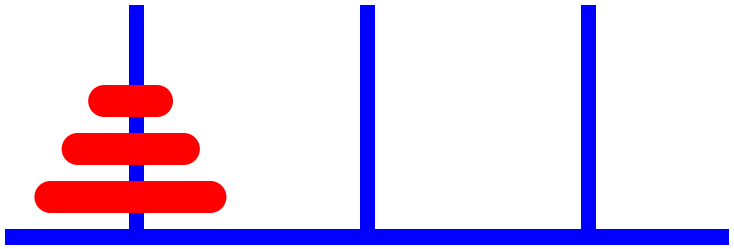


Towers of Hanoi: Description

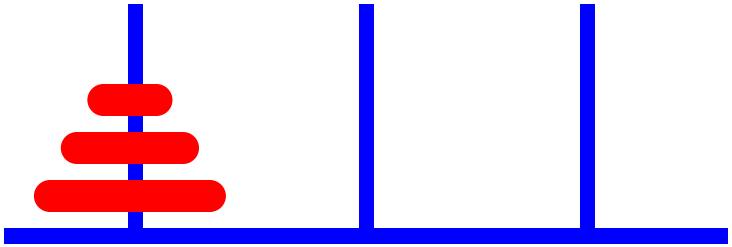
Goal: Move entire tower to another peg

Rules:

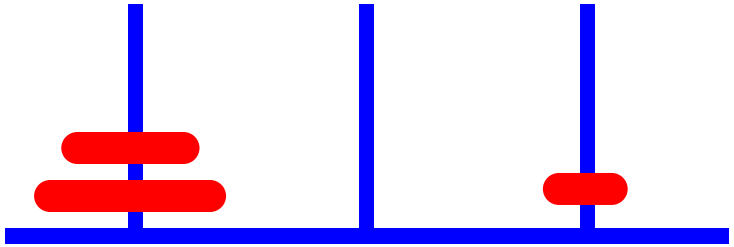
1. You can move only the top disk from a peg.
2. You can only put a smaller on a larger disk (or on an empty peg)



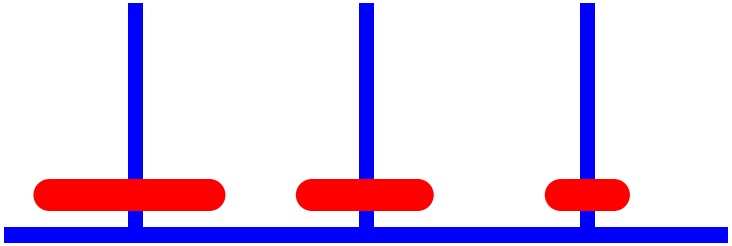
Towers of Hanoi



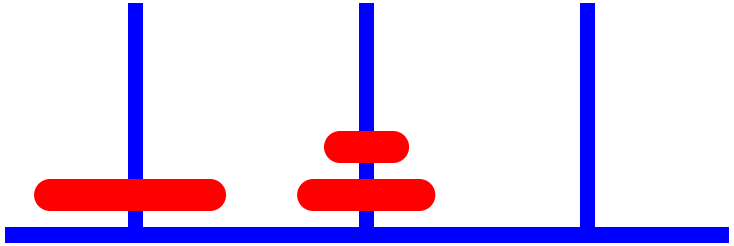
Original Configuration



Move 1

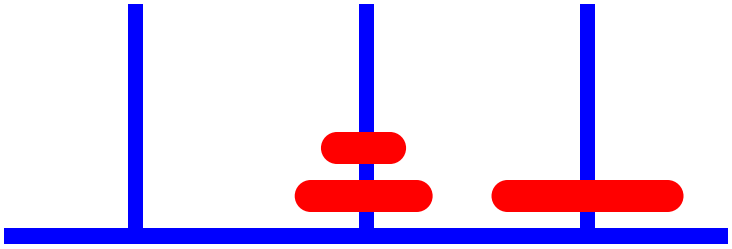


Move 2

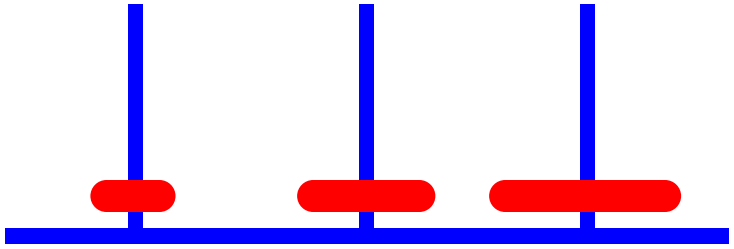


Move 3

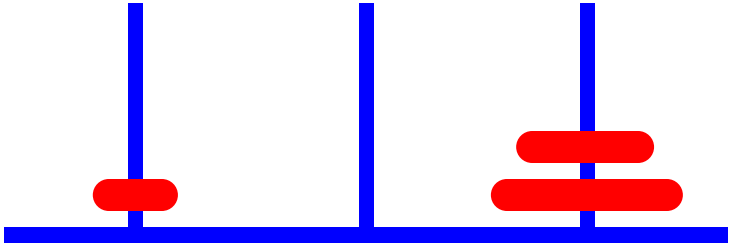
Towers of Hanoi



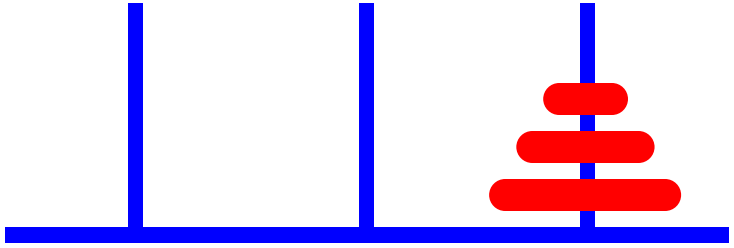
Move 4



Move 5



Move 6



Move 7 (done)

Towers of Hanoi: Program Specification

TABLE 7.1

Inputs and Outputs for Towers of Hanoi Problem

Problem Inputs
Number of disks (an integer)
Letter of starting peg: L (left), M (middle), or R (right)
Letter of destination peg (L, M, or R), but different from starting peg
Letter of temporary peg (L, M, or R), but different from starting peg and destination peg
Problem Outputs
A list of moves

Towers of Hanoi: Program Specification (2)

TABLE 7.2

Class `TowersOfHanoi`

Method	Behavior
`public String showMoves(int n, char startPeg, char destPeg, char tempPeg)`	Builds a string containing all moves for a game with `n` disks on `startPeg` that will be moved to `destPeg` using `tempPeg` for temporary storage of disks being moved.

Towers of Hanoi: Recursion Structure

```
move(n, src, dst, tmp) =
   if n == 1:  move disk 1 from src to dst
   otherwise:
       move(n-1, src, tmp, dst)
       move disk n from src to dst
       move(n-1, tmp, dst, src)
```

Lecture content adapted from Michael T. Goodrich textbook, chapters 3.